Übungsblatt 1: Datenrepräsentation & Logik-Refresher

Mikrocomputertechnik 1

Musterlösung

Zielsetzung

In diesem ersten Übungsblatt festigen Sie das theoretische Fundament der Computerarchitektur. Sie wandeln Zahlen zwischen verschiedenen Zahlensystemen um, bilden negative Zahlen im Zweierkomplement, analysieren die Speicherausrichtung (Endianness) und wiederholen die Grundlagen kombinatorischer Schaltungen.

Aufgabe 1: Zahlensysteme und Zweierkomplement

Die RISC-V-Architektur rechnet intern ausschließlich mit Nullen und Einsen. Ein tiefes Verständnis der Datenrepräsentation ist für die spätere Assembler-Programmierung essenziell.

Dezimal zu Binär und Hexadezimal

Wandeln Sie die Dezimalzahl 85 in eine 8-Bit-Binärzahl und in eine Hexadezimalzahl um.

Das Zweierkomplement

Bei RISC-V gibt es kein separates Vorzeichen-Bit wie bei Sign-Magnitude. Negative Zahlen werden im Zweierkomplement dargestellt. Bilden Sie das 8-Bit-Zweierkomplement der Dezimalzahl -85. Zeigen Sie Ihre Rechenschritte (Invertieren und $+1$).

Ihre Lösung

(Notieren Sie Ihre Rechenwege hier.)

Musterlösung Aufgabe 1

Zu 1.1:

- Binär: $85 = 64 + 16 + 4 + 1 \rightarrow$ `0101 0101`

- Hexadezimal: Nibble aufteilen → `0101` (5) und `0101` (5) → `0x55`

Zu 1.2 (−85 im Zweierkomplement):

1. Positive Binärzahl: `0101 0101`
2. Alle Bits invertieren: `1010 1010`
3. Eins addieren: `1010 1011`
4. Ergebnis (−85): `1010 1011` (hexadezimal: `0xAB`)

Aufgabe 2: Speicherausrichtung (Endianness)

Stellen Sie sich vor, Sie speichern das 32-Bit-Datenwort `0xAABBCCDD` in den Arbeitsspeicher. Die Startadresse (Basisadresse) lautet `0x1000`. Der Arbeitsspeicher ist byte-adressierbar (jedes Byte bekommt eine eigene Adresse).

Wie sehen die Inhalte der Speicherzellen von Adresse `0x1000` bis `0x1003` in einer Little-Endian-Architektur (wie RISC-V) und in einer Big-Endian-Architektur aus? Füllen Sie die Tabelle aus.

Speicheradresse	Little-Endian (RISC-V)	Big-Endian
`0x1000`		
`0x1001`		
`0x1002`		
`0x1003`		

Ihre Lösung

(Füllen Sie die Tabelle aus.)

Musterlösung Aufgabe 2

Bei Little-Endian wird das Least Significant Byte (hier `DD`) an der niedrigsten Speicheradresse abgelegt. Big-Endian legt das Most Significant Byte (`AA`) zuerst ab.

Speicheradresse	Little-Endian (RISC-V)	Big-Endian
`0x1000`	`DD`	`AA`
`0x1001`	`CC`	`BB`
`0x1002`	`BB`	`CC`
`0x1003`	`AA`	`DD`

Aufgabe 3: Kombinatorik (Der Multiplexer)

Der Multiplexer (MUX) ist die Weiche unseres Datenpfads. Ein 2-zu-1-Multiplexer hat zwei Dateneingänge (A und B), einen Steuereingang (S für Select) und einen Ausgang (Y).

Regel:

- Wenn $S = 0$, wird Eingang A durchgeschaltet ($Y = A$).

- Wenn $S = 1$, wird Eingang B durchgeschaltet ($Y = B$).

Füllen Sie die vollständige Wahrheitstabelle für dieses Bauteil aus.

S (Select)	A	B	$\rightarrow$	Y (Ausgang)
0	0	0		
0	0	1		
0	1	0		
0	1	1		
1	0	0		
1	0	1		
1	1	0		
1	1	1		

Ihre Lösung

(Tragen Sie die Ausgangswerte Y in die Tabelle ein.)

Musterlösung Aufgabe 3

S (Select)	A	B	$\rightarrow$	Y (Ausgang)	Erklärung
0	0	0		0	$S = 0$ wählt A ($A = 0$)
0	0	1		0	$S = 0$ wählt A ($A = 0$)
0	1	0		1	$S = 0$ wählt A ($A = 1$)
0	1	1		1	$S = 0$ wählt A ($A = 1$)
1	0	0		0	$S = 1$ wählt B ($B = 0$)
1	0	1		1	$S = 1$ wählt B ($B = 1$)
1	1	0		0	$S = 1$ wählt B ($B = 0$)
1	1	1		1	$S = 1$ wählt B ($B = 1$)

Aufgabe 4: Architektur-Verständnis

Erklären Sie in zwei bis drei kurzen Sätzen den fundamentalen Unterschied zwischen kombinatorischer Logik (z. B. ALU oder Multiplexer) und sequenzieller Logik (z. B. D-Flip-Flop oder Register File). Welches Bauteil benötigt zwingend ein Taktsignal (Clock)?

Ihre Lösung

(Notieren Sie Ihre Antwort hier.)

Musterlösung Aufgabe 4

- Kombinatorische Logik hat kein Gedächtnis. Der Ausgang ändert sich unmittelbar, wenn sich die Eingänge ändern. Es wird kein Takt (Clock) benötigt.

- Sequenzielle Logik besitzt ein Gedächtnis (Speicher). Der Zustand (Ausgang) ändert sich nicht sofort, sondern erst bei einer auslösenden Taktflanke (Clock). Flip-Flops und das Register File benötigen daher zwingend ein Taktsignal.

Aufgabe 5: Volladdierer (Paper & Pencil)

Ein 1-Bit-Volladdierer hat Eingänge A, B, C_{in} und Ausgänge S (Summe) sowie C_{out} (Carry).

Wahrheitstabelle

Füllen Sie die Wahrheitstabelle für $A = 1$, $B = 1$, $C_{\text{in}} = 0$ und für $A = 1$, $B = 1$, $C_{\text{in}} = 1$ aus (je S und C_{out}).

A	B	C_{in}	S	C_{out}
1	1	0		
1	1	1		

4-Bit-Addition

Addieren Sie per Hand (binär, mit Carry-Kette) die 4-Bit-Zahlen $A = 0110_2$ und $B = 0101_2$. Geben Sie Summe und finalen Carry an.

Ihre Lösung

(Notieren Sie Ihre Rechenwege hier.)

Musterlösung Aufgabe 5

Wahrheitstabelle:

A	B	C_{in}	S	C_{out}
1	1	0	0	1
1	1	1	1	1

4-Bit-Addition $0110_2 + 0101_2$:

- Bit 0: $0 + 1 = 1$, Carry 0
- Bit 1: $1 + 0 + 0 = 1$, Carry 0
- Bit 2: $1 + 1 + 0 = 0$, Carry 1
- Bit 3: $0 + 0 + 1 = 1$, Carry 0
- Summe $= 1011_2 = 11_{10}$, finaler Carry $= 0$